

Mutable Idle Timeout?

QUIC – IETF 126 – 2026-07

Lucas Pardue – lucas@lucaspardue.com



Life has a very simple plot: first
you're here and then you're not.

— *Eric Idle* —

AZ QUOTES

QUIC v1 Idle Timeout

[https://datatracker.ietf.org/doc/html/rfc9000#section-10.1\](https://datatracker.ietf.org/doc/html/rfc9000#section-10.1)

If a `max_idle_timeout` is specified by **either endpoint** in its transport parameters ([Section 18.2](#)), the connection is **silently closed** and its state is discarded when it remains idle for longer than the **minimum of the `max_idle_timeout`** value advertised by both endpoints.

Each endpoint advertises a `max_idle_timeout`, but the effective value at an endpoint is computed as the minimum of the two advertised values (or the sole advertised value, if only one endpoint advertises a non-zero value). By announcing a `max_idle_timeout`, **an endpoint commits to initiating an immediate close** ([Section 10.2](#)) if it abandons the connection prior to the effective value.

Client

`max_idle_timeout = 30,000`

Server

`max_idle_timeout = 60,000`

Effective idle timeout = $\min(60000, 30000) = 30,000$

Client

`max_idle_timeout = 30,000`

Server

`max_idle_timeout = 0`

Effective idle timeout = 30,000

What values are common?

Implementation	Default (ms)
Chromium	30,000
quic-go	30,000
MsQuic	30,000
quinn	30,000
picoquic	30,000
lsquic	30,000
neqo	30,000
haproxy	30,000
h2o	30,000
aioquic	60,000

Implementation	Default (ms)
mvfst	60,000
GFE	240,000
envoy	300,000
ngtcp2	0
s2n-quic	None
quicly	None
quiche (Cloudflare)	none

Assertions

1. max_idle_timeout is unique across all TPs
 - a. enables an actual negotiation that applies to both endpoints for sending and receiving packets
2. Clients can advertise multiple ALPNs - same idle timeout for all
3. A single application protocol can have many use cases, only established after handshake
4. An application protocol can have different lifecycle events e.g. graceful close
5. An application protocol can have additional trust model / authentication mechanisms
 - a. Basis to control resource allocation

HTTP/3 GOAWAY

<https://datatracker.ietf.org/doc/html/rfc9114#connection-shutdown>

Endpoints initiate the graceful shutdown of an HTTP/3 connection by sending a GOAWAY frame.

...

Endpoints **MUST NOT** initiate new requests or promise new pushes on the connection after receipt of a GOAWAY frame from the peer. Clients **MAY** establish a new connection to send additional requests.

HTTP/3 GOAWAY cont'd

- Once a server send GOAWAY, the only connection driver is sending HTTP response data
- Server's are likely to implement own graceful close period
- Eventually the idle timeout or explicit close will happen

Would it be an improvement to set a shorter idle timeout in this phase of connection?

For example, 30 seconds is common. Perhaps an idle timeout of 5 seconds could allow for aggressive tidyup without needing network traffic.

draft-pardue-quit-idle-timeout-update

<https://www.ietf.org/archive/id/draft-pardue-quit-idle-timeout-update-00.html>

Adds 3 new frames:

- IDLE_TIMEOUT_UPDATE_REQUEST
- IDLE_TIMEOUT_UPDATE_ACCEPT
- IDLE_TIMEOUT_UPDATE_REJECT

Either peer can request an update to the effective idle timeout.

Accepting the request switches it. Rejecting it causes no change.

Value of 0 will disable the idle timeout mechanism

Questions

1. Would mutable idle timeouts solve a **concrete problem**?
2. Would updating **other** TPs solve **other** problems?
3. By extension: would a generic mechanism be better than a per-TP one?